Notes for C++

Section 4 : Loops

The while loop

Here is a program that repeats the same instruction several times:

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
}
```

We can achieve the same effect in the next program by using a while loop:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  counter = 0;
  while (counter < 5) {
    cout << "Hello\n";
    counter++;
  }
}
```

Notice that there is now a variable (there wasn't one before). This is the ***loop variable*** and counts how many times the loop happens.

The syntax of the while loop (or how to write a while loop)

- it starts with the reserved word while
- there is a ***condition*** (or a test) inside ordinary brackets (parentheses)
- there are statements inside the ***body*** of the loop – inside the curly brackets

The semantics of the while loop (or how does it work?)

Step 1: do the test. If the test is true, do Step 2. If the test is false, ignore the while loop.

Step 2: execute the instructions in the body of the while loop.

Step 3: when you reach the end of the loop (the last closing curly bracket), go back to Step 1.

Walkthrough (also known as a desk check)

Work through the program, doing exactly what the computer would do. Keep track of all changes to any of the variables and also show what will be displayed on the screen. In this example, we keep track of the variable called counter as follows. You can work out what the display would look like.

counter ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 5

Notice how we ***cross out*** previous values so we can work backwards – if needed to solve an error.

Example : A program to display the numbers from 1 to 10

```
#include <iostream>
using namespace std;

int number;

int main() {
  number = 1;                    // initialise the loop variable
  while (number <= 10) {
    cout << number << endl;
    number = number + 1;         // can we use a shortcut here?
  }
}
```

The ***loop variable*** is the variable that controls the loop. In this program, `number` is the loop variable. It is important to load a value into the loop variable before the loop starts – this is called ***initialising the loop variable***. For homework = modify the program to display numbers from 38 to 54.

Example : A program to keep a running total

We keep a ***running total*** by adjusting the total after each number in the list, as shown in this program to add up the numbers from 3 to 8:

```
#include <iostream>
using namespace std;

int number, total;

int main() {
  total = 0;
  number = 3;
  cout << "Add up the numbers from 3 to 8.\n";
  while (number <= 8) {
    total = total + number;
    number = number + 1;
  }
  cout << "The total is " << total << endl;
}
```

It is important to initialise the total to zero before starting the loop. Running totals must always start at zero. Some people may tell you that variables (including totals) are always set to zero before they are used. This is correct for some variables but not for all variables (more on this later). It is a good habit to always initialise all totals to zero before using them. For homework = modify the program to add up the numbers from 87 to 93. Predict what the total will be before running the program.

Example : A program to enter numbers and keep a running total

We can enter several numbers and keep a running total as shown in this program to add up four numbers:

```
#include <iostream>
using namespace std;

int count, number, total;

int main() {
  total = 0;
  count = 0;
  cout << "Enter four numbers to be added together:\n";
  while (count < 4) {
    cout << "Enter a number ";
    cin >> number;
    total = total + number;
    count++;                        // using a shortcut here
  }
  cout << "The total is " << total << endl;
}
```

We initialise total to zero (as we always do for total variables). We do not need to initialise number as it receives a value that is entered so there is no need to initialise it. We initialise variables that are going to have something added to them (such as total) or are going to be used in a test (such as count). We do not need to initialise variables that are simply going to be loaded with a value later on.

The loop variable in this program is `count`. It keeps track of how many numbers have been entered. We initialise `count` to zero to indicate that, at the start of the program, we have entered zero numbers. For homework = modify this program to read in seven numbers.

Example : A program that uses a trailing value.

The previous program was designed to read in 4 numbers. What if you do not know how many numbers are going to be entered? One way to solve this problem is to use a ***trailing value***. A trailing value is entered after the other numbers to indicate that the data entry has ended. It is important that the trailing value **is not a valid data item**. For example, if we had to enter people's ages, then a good trailing value would be -1 because -1 is not a valid age. -1 is a good trailing value for all sorts of data including student IDs, phone numbers, bank account numbers, etc because it is usually invalid data. However there are some situations where -1 is valid data and (in those situations) you would not be able to use -1 as a trailing value. Here is the program to enter any amount of numbers – make sure you enter -1 after the numbers because we have selected -1 to be the trailing value:

Version 1 of the program that uses a trailing value (this version has some problems)

```
#include <iostream>
using namespace std;

/* a program to read in positive numbers and add them together */
```

```
int number, total;

int main() {
  total = 0;
  number = 999;
  while (number >= 0) {
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
    total = total + number;
  }
  cout << "The total is " << total << endl;
}
```

We notice two things about this program:

- number is initialised to 999. This is because number is used in a test ***before*** it has been entered. Never use a variable in a test before it has been loaded with a value because it is not possible to predict the result of the test. In order to force the test to be true, we stick any value into number. I selected 999 but you could use any positive integer value.
- the program produces incorrect results (did you test it)? This is because the -1 is added to the total. This is exactly what **NOT** to do with a trailing value – it must **NOT** be treated as data.

Let us write a new version of the program to work in a better way:

Version 2 of the program that uses a trailing value (this version works correctly)

```
#include <iostream>
using namespace std;

/* a program to read in numbers and add them together */

int number, total;

int main() {
  total = 0;
  cout << "Enter a number (-1 to stop) ";
  cin >> number;
  while (number >= 0) {
    total = total + number;
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
  }
  cout << "The total is " << total << endl;
}
```

Interesting things to notice in this program:

- if we enter the first value **before** the loop, we don't need 999; we deal only with actual values
- we now need two places where we enter data – once before the loop and again inside the loop
- we have to change the order of the instructions in the loop – why?
- initialising can be done by reading in a value as well as by assigning a value

Another example of a program that uses a loop: The Fibonacci sequence

The Fibonacci sequence appears in nature and is: 0 1 1 2 3 5 8 13 21 …

```
#include <iostream>
using namespace std;

int a, b, c;

int main() {
  a = 0;
  b = 1;
  c = 0;
  cout << "0 1";
  while (c < 200) {   // decide where to stop
    c = a + b;
    cout << " " << c;
    a = b;
    b = c;
  }
}
```

For more interesting information on the Fibonacci sequence:
https://en.wikipedia.org/wiki/Fibonacci_number

Example : Making a program run many times

Loops can be useful to make the task of a program happen repeatedly until the user wishes to stop. For example, a user running a program that reads in a year and prints out whether or not it is a leap year, might want to check several years. We can write a loop that stops when a user enters a specified ending value (a trailing value).

```
#include <iostream>
using namespace std;

int year;
bool leapyear;

int main() {
  cout << "Enter a year (-1 to stop) ";
  cin >> year;
  while (year >= 0) {
    leapyear = false;
    if ((year % 4 == 0) && ((year % 100 != 0) || (year % 400 == 0))) {
      leapyear = true;
    }
    if (leapyear) {
      cout << year << " is a leap year.\n";
    } else {
      cout << year << " is NOT a leap year.\n";
    }
    cout << "Enter a year (-1 to stop) ";
    cin >> year;
  }
}
```

Interesting things to notice in this program:

- we use only one if-statement to check for a leap year
- we have included a loop in the original program (and changed the order of some instructions) in order to repeat the program many times
- decide how to stop the program – in this case we stop if the year is negative

Data validation

In a course, students write two tests. Read in the two test marks for a student and check that each test mark is from 0 to 50. Then add the test marks to calculate the final result.

Version 1 : keep entering the data until it is correct

```
#include <iostream>
using namespace std;

/* read in two test marks (each out of 50) and check for valid data */
/* add the test marks to obtain the final result */

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  while ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
    cout << "Enter the mark for test 1 (0 to 50): ";
    cin >> mark1;
  }

  cout << "Enter the mark for test 2 (0 to 50): ";
  cin >> mark2;
  while ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    cout << "Enter the mark for test 2 (0 to 50): ";
    cin >> mark2;
  }
  result = mark1 + mark2;
  cout << "The final result is " << result;
}
```

Interesting things to notice in this program:

- we want to validate (check) that the correct data has been entered
- a loop is included for each data item that will be entered
- if the data is correct, the user will not notice anything (the loops will not run)
- an error message is printed if the data is not in the correct range and the user must enter the data again – this continues until the user enters valid data
- these loops ensure that the calculation only happens on valid data
- we use OR (`||`) to check for values that are outside a valid range

Version 2 : if data is not correct then stop the program

```
#include <iostream>
```

```
#include <cstdlib>          // we use exit from cstdlib
using namespace std;

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  if ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
    cout << "Program will now close";
    exit(10);
  }
  cout << "Enter the mark for test 2 (0 to 50): ";
  cin >> mark2;
  if ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    cout << "Program will now close";
    exit(20);
  }
  result = mark1 + mark2;
  cout << "The final result is " << result;
}
```

Interesting things to notice in this program:

- `exit` immediately stops the program
- it doesn't matter what number you put inside the brackets for `exit`
- there is no opportunity to enter the data again
- the user is forced to rerun the program if they enter invalid data
- stopping the program for invalid input means the calculation only happens on valid data

Version 3 : use if's and else's to avoid calculating with invalid data

```
#include <iostream>
using namespace std;

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  if ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
  } else {
    cout << "Enter the mark for test 2 (0 to 50): ";
    cin >> mark2;
    if ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    } else {
      result = mark1 + mark2;
      cout << "The final result is " << result;
    }
  }
```

```
}
```

Which data validation technique should you use?

Allow the data to be entered again (version 1) if the data is simple, e.g. test marks. It is really annoying if the whole program stops just because you typed one mark incorrectly.

Stop the program if the data is really complicated (version 2). For example, if the data was a long list of specifications for an engineering project. If complicated data is not entered correctly it may be better to stop the program.

Avoid version 3 as the if's and else's are very hard to read and it is easy to make a mistake in your program.

The for loop

A loop very often contains:

- *initialising* of the loop variable (the variable that 'controls' the loop)
- a *condition* or *test* that involves the loop variable
- a *change* to the loop variable (e.g. m = m + 3;). For example:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  counter = 0;            /* initialising the loop variable */
  while (counter < 5) {   /* the condition */
    cout << "Hello\n";
    counter++;            /* a change to the loop variable */
  }
}
```

In a **for loop** the three parts of the loop (listed above) are collected together inside the brackets. For the example above, we can change the **while loop** to a **for loop** as follows:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  for (counter = 0; counter < 5; counter++) {
    cout << "Hello\n";
  }
}
```

Note how the initialising, the condition and the change now all appear inside the brackets.

Example: A program to find the average (or mean) of 4 numbers

```
#include <iostream>
using namespace std;

int counter;
float number, total, average;

int main() {
  total = 0.0;
  for (counter = 0; counter < 4; counter++) {
    cout << "Enter a number ";
    cin >> number;
    total = total + number;
  }
  average = total / 4.0;
  cout << "The average is " << average;
}
```

Interesting things to notice in this program:

- there is only one variable called `number` but there are many numbers
- note the semi-colons between the three parts of the for loop (not commas)

Example: A program to calculate final results in an exam

```
#include <iostream>
using namespace std;

// there are 8 questions in an exam
// if you get zero for 4 or more questions - you fail
// otherwise you pass if your total mark is >= 50

int count, zeromark;
float mark, total;

int main() {
  total = 0.0;
  zeromark = 0;
  for (count = 0; count < 8; count++) {
    cout << "Enter mark for Question " << count+1 << ": ";
    cin >> mark;
    total = total + mark;
    if (mark == 0.0) {
      zeromark++;
    }
  }
  if (zeromark >= 4) {
    cout << "FAIL - you got zero for " << zeromark << " questions";
  } else {
    if (total < 50.0) {
      cout << "FAIL - you got a total of " << total;
    } else {
      cout << "PASS - you got a total of " << total;
    }
```

```
  }
}
```

Note how count starts at 0 and changes by 1 until it gets up to 8.

Does this program work correctly? Do a ***walkthrough*** to find out.

Incrementing and decrementing

Adding 1 to a variable is called **incrementing**. It can be written as `k = k + 1` or `k += 1` or `k++`

Subtracting 1 from a variable is **decrementing**. It can be written as `k = k - 1` or `k -= 1` or `k--`

Remember that the **change** in a for loop does not always have to be increasing or decreasing by 1.

For example: `k = amount` `k = k * 3` `k = j % 2` `k = k - 5`

When to use a while loop and when to use a for loop?

Use a for loop when you know exactly how many times the loop will happen. Use a while loop in any other situation. This is not a definite rule but is a useful guideline to remember.

Example : A program to find the biggest number (the maximum)

```
#include <iostream>
using namespace std;

int biggest, number;

int main() {
  cout << "Enter the first number (-1 to stop) ";
  cin >> number;
  biggest = number;
  while (number > 0) {
    if (biggest < number) {
      biggest = number;
    }
    /* put an extra cout statement here */
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
  }
  cout << "The biggest number is " << biggest;
}
```

Interesting things to notice in this program:

- we use a while loop and not a for loop – why not?
- the loop variable `number` is initialised by reading it before the loop
- the variable `biggest` is initialised to the first number – why do we do this?
- can you modify this program to find the smallest number (minimum)?

Extra cout statements for debugging

The use of temporary, debugging `cout` statements is always useful. In this program, you could display the values of `biggest` and `number` inside the loop and follow the progress of the program.

Replace the comment `/* put an extra cout statement here */`
with: `cout << "Number is << number << " and max is " << biggest;`

IMPORTANT – the concept of the digits of an integer

An integer number consists of one or more digits. For example, the number 485 consists of 3 digits which are 4, 8, and 5. To 'break' a number into the individual digits, we need a loop, a remainder calculation and an integer division. For example, let us look at reversing a number:

A program to reverse an integer

For example: if the number is 4072 then the reverse of the number is 2704

```
#include <iostream>
using namespace std;

int number, reverse, digit, temp;

int main() {
  cout << "Enter an integer ";
  cin >> number;
  temp = number;
  reverse = 0;
  while (temp > 0) {
    digit = temp % 10;
    reverse = reverse * 10 + digit;
    temp = temp / 10;
  }
  cout << "Number is " << number;
  cout << " and the reverse is " << reverse;
}
```

Does this program work correctly? Do a ***walkthrough*** to find out.

Here is an example of a walkthrough for this program. Make a list of the variables and then work through the program, one instruction at a time. When a variable changes value, cross it out and write the new value next to the variable name.

`number = 3851`

`reverse =` ~~`0`~~ ~~`1`~~ ~~`15`~~ ~~`158`~~ `1583`

`digit =` ~~`1`~~ ~~`5`~~ ~~`8`~~ `3`

`temp =` ~~`3851`~~ ~~`385`~~ ~~`38`~~ ~~`3`~~ `0`

Usually you would also keep track of what is displayed on the screen at each point. However, in this program there is only one `cout` statement at the end of the program. For this particular walkthrough, the following would be displayed:

```
Enter an integer 3851
Number is 3851 and the reverse is 1583
```

Infinite loops

Make sure that a loop will actually end. A loop that never ends is called an infinite loop.
For example, this loop never ends – why not? What will happen when you run it?

```
#include <iostream>
using namespace std;

int count;

int main() {
  count = 0;
  while (count < 5) {
    cout << "Hello";
  }
}
```

A different approach to loops (while true loops)

It is possible to set up an infinite loop and then use the **break statement** to stop the loop.
For example:

```
#include <iostream>
using namespace std;

int number, total;

/* a program to read in numbers and add them together */

int main() {
  total = 0;
  while (true) {
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
    if (number < 0) {
      break;
    }
    total = total + number;
  }
  cout << "The total is " << total << endl;
}
```

Interesting things to notice in this program:

- this is an infinite loop because the test is always true!
- break stops a loop (moves the processor on to the code after the loop)
- usually break will be inside an if statement
- no need for more than one `cin`, because no loop variable to be initialised before the loop
- the use of the "while true" loop makes the program simpler and easier to read

This program uses the opposite test to the way we previously wrote this program.

The previous program used the test: `while (number >= 0) {`

This program uses the opposite test: `if (number < 0) {`

This is because the standard while loop is asking the test question “must the loop run again?” whereas the if statement inside the “while true” loop is asking the opposite test question which is “must the loop stop?”